

Vorlesung

Theoretische Informatik

(09 Eigenschaften (deterministisch) kontextfreier Sprachen, PDA, DPDA, Parser)

Alois Heinz
Hochschule Heilbronn,
Max-Planck-Str. 39, 74081 Heilbronn
`heinz@hs-heilbronn.de`

05. Mai/12. Juni 2026

Nichtdeterministische Kellerautomaten

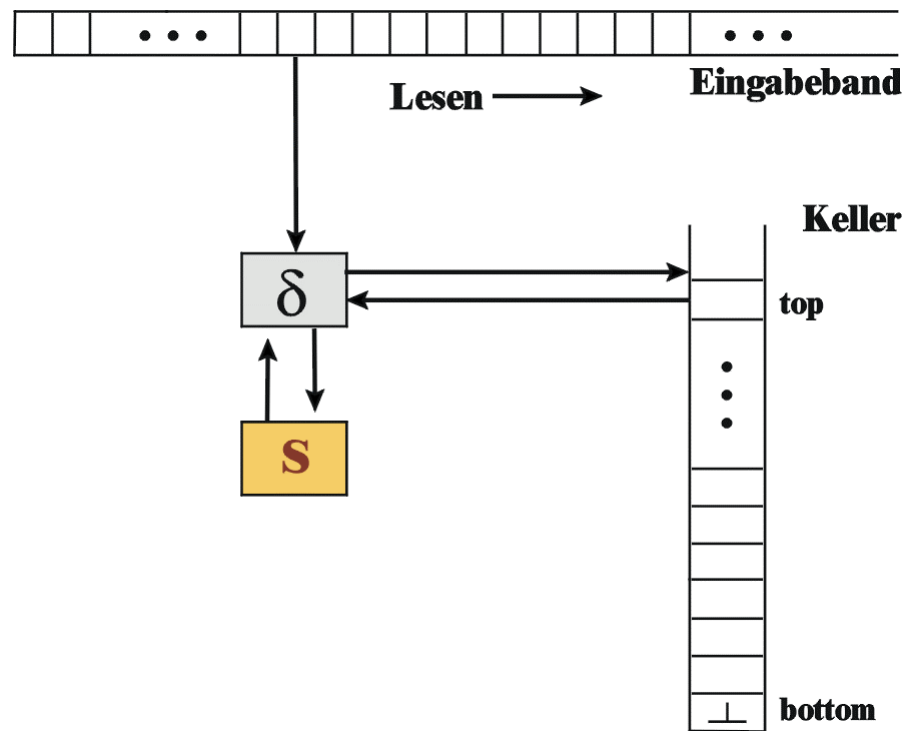
Ein (nichtdeterministischer) Kellerautomat (PDA) ist ein Septupel

$$K = (\Sigma, S, \Gamma, \delta, s_0, \perp, F)$$

Dabei ist Σ das Eingabealphabet, S die endliche Zustandsmenge, Γ das Kelleralphabet,

$$\delta : S \times (\Sigma \cup \{\epsilon\}) \times \Gamma \rightarrow 2^{S \times \Gamma^*}$$

die Zustandsüberföhrungsfunktion, $s_0 \in S$ der Startzustand, $\perp \in \Gamma$ das Keller-Bottomsymbol und $F \subseteq S$ die Menge der Endzustände.



Konfigurationsübergänge eines PDA

Eine Konfiguration

$$k = (s, w, \alpha) \in S \times \Sigma^* \times \Gamma^*$$

enthält den aktuellen Zustand s , das zu verarbeitende Suffix w des Eingabewortes, und den aktuellen Kellerinhalt α

Konfigurationsübergänge sind gegeben durch die Relation

$$\vdash \subseteq (S \times \Sigma^* \times \Gamma^*) \times (S \times \Sigma^* \times \Gamma^*)$$

so dass $(s, av, A\alpha) \vdash (s', v, \beta\alpha)$ genau dann, wenn $(s', \beta) \in \delta(s, a, A)$ mit $s, s' \in S$, $a \in \Sigma \cup \{\epsilon\}$, $\alpha, \beta \in \Gamma^*$, $A \in \Gamma$

Die vom Kellerautomaten $K = (\Sigma, S, \Gamma, \delta, s_0, \perp, F)$ mit Endzustand erkannte Sprache über Σ ist

$$L_F(K) = \{w \in \Sigma^* \mid (s_0, w, \perp) \vdash^* (s_f, \epsilon, \gamma), s_f \in F, \gamma \in \Gamma^*\}$$

Mit PDA_Σ^F wird die Klasse der von PDA mit Endzustand akzeptierten Sprachen bezeichnet.

Beispiel für einen PDA

Ein Kellerautomat soll $L_1 = \{a^n b^n \mid n \geq 0\}$ erkennen:

$$K_1 = (\{a, b\}, \{s_0, s_1, s_f\}, \{A, \perp\}, \delta_1, s_0, \perp, \{s_f\}) \quad \text{mit}$$

$$\begin{aligned} \delta_1 = & \{(s_0, \epsilon, \perp, \{(s_f, \epsilon)\}), \\ & (s_0, a, \perp, \{(s_0, A\perp)\}), \\ & (s_0, a, A, \{(s_0, AA)\}), \\ & (s_0, b, A, \{(s_1, \epsilon)\}), \\ & (s_1, b, A, \{(s_1, \epsilon)\}), \\ & (s_1, \epsilon, \perp, \{(s_f, \epsilon)\})\} \end{aligned}$$

Das Wort $w = aaabbb$ wird erkannt:

$$\begin{aligned} (s_0, aaabbb, \perp) \vdash (s_0, aabbb, A\perp) \vdash (s_0, abbb, AA\perp) \vdash (s_0, bbb, AAA\perp) \vdash \\ (s_1, bb, AA\perp) \vdash (s_1, b, A\perp) \vdash (s_1, \epsilon, \perp) \vdash (s_f, \epsilon, \epsilon) \end{aligned}$$

Der Keller dient zum Merken der gelesenen a .

Beispiel für einen deterministischen PDA

Ein Kellerautomat soll $L_2 = \{w c \tilde{w} \mid w \in \{a, b\}^*\}$ erkennen
($\tilde{w}$ ist das gespiegelte w):

$$K_2 = (\{a, b, c\}, \{s_0, s_c, s_f\}, \{A, B, \perp\}, \delta_2, s_0, \perp, \{s_f\}) \quad \text{mit}$$

$$\begin{aligned} \delta_2 = \{ & (s_0, c, \perp, \{(s_f, \epsilon)\}), & (\text{wenn Eingabe nur } c) \\ & (s_0, a, \perp, \{(s_0, A\perp)\}), & (\text{erstes Zeichen } a \text{ merken}) \\ & (s_0, b, \perp, \{(s_0, B\perp)\}), & (\text{erstes Zeichen } b \text{ merken}) \\ & (s_0, a, A, \{(s_0, AA)\}), & (\text{weitere } a \text{ merken}) \\ & (s_0, a, B, \{(s_0, AB)\}), & (\text{weitere } a \text{ merken}) \\ & (s_0, b, A, \{(s_0, BA)\}), & (\text{weitere } b \text{ merken}) \\ & (s_0, b, B, \{(s_0, BB)\}), & (\text{weitere } b \text{ merken}) \\ & (s_0, c, A, \{(s_c, A)\}), & (c \text{ lesen, Keller bleibt}) \\ & (s_0, c, B, \{(s_c, B)\}), & (c \text{ lesen, Keller bleibt}) \\ & (s_c, a, A, \{(s_c, \epsilon)\}), & (\text{Spiegel-}a \text{ lesen}) \\ & (s_c, b, B, \{(s_c, \epsilon)\}), & (\text{Spiegel-}b \text{ lesen}) \\ & (s_c, \epsilon, \perp, \{(s_f, \epsilon)\}) \} & (\text{Keller leer}) \end{aligned}$$

Beispiel für einen echt nichtdeterministischen PDA

Ein Kellerautomat soll $L_3 = \{w \tilde{w} \mid w \in \{a, b\}^*\}$ erkennen
($\tilde{w}$ ist das gespiegelte w):

$$K_3 = (\{a, b\}, \{s_0, s_c, s_f\}, \{A, B, \perp\}, \delta_3, s_0, \perp, \{s_f\}) \quad \text{mit}$$

$$\begin{aligned} \delta_3 = \{ & (s_0, \epsilon, \perp, \{(s_f, \epsilon)\}), & (w = \epsilon \text{ wird akzeptiert}) \\ & (s_0, a, \perp, \{(s_0, A\perp)\}), & (\text{erstes Zeichen } a \text{ merken}) \\ & (s_0, b, \perp, \{(s_0, B\perp)\}), & (\text{erstes Zeichen } b \text{ merken}) \\ & (s_0, a, B, \{(s_0, AB)\}), & (a \text{ merken, wenn vorher } b) \\ & (s_0, b, A, \{(s_0, BA)\}), & (b \text{ merken, wenn vorher } a) \\ & (s_0, a, A, \{(s_0, AA), (s_c, \epsilon)\}), & (a \text{ nach } a \text{ merken, oder Spiegel-}a \text{ lesen}) \\ & (s_0, b, B, \{(s_0, BB), (s_c, \epsilon)\}), & (b \text{ nach } b \text{ merken, oder Spiegel-}b \text{ lesen}) \\ & (s_c, a, A, \{(s_c, \epsilon)\}), & (\text{Spiegel-}a \text{ lesen}) \\ & (s_c, b, B, \{(s_c, \epsilon)\}), & (\text{Spiegel-}b \text{ lesen}) \\ & (s_c, \epsilon, \perp, \{(s_f, \epsilon)\}) \} & (\text{Keller leer, Wort akzeptiert}) \end{aligned}$$

Bem: $w \in L(K_3)$, wenn es eine akzeptierende Konfigurationsfolge für w gibt.

Akzeptieren mit leerem Keller

Sei $K = (\Sigma, S, \Gamma, \delta, s_0, \perp, F)$ ein Kellerautomat.

Die von K mit leerem Keller erkannte Sprache über Σ ist

$$L_\epsilon(K) = \{w \in \Sigma^* \mid (s_0, w, \perp) \vdash^* (s, \epsilon, \epsilon), s \in S\}$$

Mit PDA_Σ^ϵ wird die Klasse der von PDA mit leerem Keller akzeptierten Sprachen bezeichnet.

Bem.: Die Angabe von F spielt bei $L_\epsilon(K)$ keine Rolle, kann daher wegfallen.

Satz. Es gilt: $PDA_\Sigma^F = PDA_\Sigma^\epsilon =: PDA_\Sigma$

Beweis. (Andeutung, Rest einfache Übung)

Zu zeigen ist eine Mengeninklusion in beiden Richtungen. Dabei wird jeweils ein PDA durch einen neuen mit eigenem Keller-Bottomsymbol simuliert:

- Sei $L \in L_F(K)$, dann gibt es einen Kellerautomaten K' mit $L \in L_\epsilon(K')$
- Sei $L \in L_\epsilon(K)$, dann gibt es einen Kellerautomaten K' mit $L \in L_F(K')$ $\square$

Äquivalenz von kontextfreien Grammatiken und PDA

Satz. Zu jedem Kellerautomaten $K = (\Sigma, S, \Gamma, \delta, s_0, \perp, F)$ kann eine kontextfreie Grammatik G angegeben werden mit $L(G) = L(K)$.

o.B.

Satz. Zu jeder kontextfreien Grammatik $G = (\Sigma, N, P, S)$ lässt sich ein Kellerautomat K_G konstruieren mit $L(K_G) = L(G)$.

Beweis. K_G wird so konstruiert, dass **Linksableitungen von G simuliert** werden:

- Ist aktuelles Eingabesymbol = Keller-Topsymbol, so wird es gelesen bzw. gelöscht.
- Ist Keller-Topsymbol = $A \in N$, so wird kein Eingabesymbol gelesen, und A wird durch α ersetzt, falls $A \rightarrow \alpha \in P$.
- Keller-Bottomsymbol ist S .

Für den entstehenden Kellerautomaten $K_G = (\Sigma, \{q\}, \Sigma \cup N, \delta, q, S, \emptyset)$ mit

$$\delta = \{(q, a, a, \{(q, \epsilon)\}) \mid a \in \Sigma\} \cup \{(q, \epsilon, A, \{(q, \alpha) \mid A \rightarrow \alpha \in P\}) \mid A \in N\}$$

gilt: $L_\epsilon(K) = L(G)$ $\square$

Bem.: Ein Zustand q reicht aus. Der Automat K_G wird **Parser von G** genannt.

Beispiel: PDA für eine kontextfreie Grammatik

Betrachte $G = (\{a, b\}, \{S\}, \{S \rightarrow aSb \mid \epsilon\}, S)$ mit $L(G) = \{a^n b^n \mid n \in \mathbb{N}\}$.

Der Parser für G ist $K_G = (\{a, b\}, \{q\}, \{a, b, S\}, \delta, q, S, \emptyset)$ mit

$$\delta = \{(q, a, a, \{(q, \epsilon)\}), (q, b, b, \{(q, \epsilon)\}), (q, \epsilon, S, \{(q, aSb), (q, \epsilon)\})\}$$

Eine akzeptierende Konfigurationenfolge für $w = aaabbb$ ist:

$$\begin{aligned} (q, aaabbb, S) &\vdash (q, aaabbb, aSb) \\ &\vdash (q, aabbb, Sb) \\ &\vdash (q, aabbb, aSbb) \\ &\vdash (q, abbb, Sbb) \\ &\vdash (q, abbb, aSbbb) \\ &\vdash (q, bbb, Sbbb) \\ &\vdash (q, bbb, bbb) \\ &\vdash (q, bb, bb) \\ &\vdash (q, b, b) \\ &\vdash (q, \epsilon, \epsilon) \end{aligned}$$

Schnitt von kontextfreien und regulären Sprachen

Es gilt nun: $TYP2_{\Sigma} = kfs_{\Sigma} = PDA_{\Sigma}$

Satz. kfs_{Σ} ist abgeschlossen unter Durchschnitt mit REG_{Σ} :

Mit $L_1 \in kfs_{\Sigma}$ und $L_2 \in REG_{\Sigma}$ gilt: $L_1 \cap L_2 \in kfs_{\Sigma}$.

Beweis. (Skizze)

- Sei K ein Kellerautomat, der L_1 mit Endzuständen erkennt und sei A ein endlicher Automat, der L_2 erkennt.
- Es wird ein Kellerautomat K' konstruiert, der K und A parallel ausführt und dann akzeptiert, wenn beide in einem Endzustand sind. $\square$

Deterministische Kellerautomaten

Ein Kellerautomat $K = (\Sigma, S, \Gamma, \delta, s_0, \perp, F)$ heißt **deterministisch** (ist ein DPDA), wenn für alle $s \in S$, $a \in \Sigma$ und $A \in \Gamma$ gilt:

$$|\delta(s, a, A)| + |\delta(s, \epsilon, A)| \leq 1$$

Eine Sprache L heißt **deterministisch kontextfrei**, wenn es einen deterministischen Kellerautomaten K gibt, der L akzeptiert:

$$L = L(K) = \{w \in \Sigma^* \mid (s_0, w, \perp) \vdash^* (s, \epsilon, \gamma), s \in F, \gamma \in \Gamma^*\}$$

Mit $DPDA_\Sigma$ wird die Klasse der Sprachen bezeichnet, die von deterministischen Kellerautomaten akzeptiert werden.

Satz. Es gilt: $DPDA_\Sigma \subset PDA_\Sigma$.

o.B.

Bsp.:

$$L_2 = \{w c \tilde{w} \mid w \in \{a, b\}^*\} \in DPDA_\Sigma$$

$$L_3 = \{w \tilde{w} \mid w \in \{a, b\}^*\} \in PDA_\Sigma, \text{ aber } L_3 = \{w \tilde{w} \mid w \in \{a, b\}^*\} \notin DPDA_\Sigma$$

Eigenschaften Deterministisch Kontextfreier Sprachen

Satz. *Das Wortproblem für eine deterministisch kontextfreie Sprache L kann in Zeit $O(n)$ entschieden werden, wenn n die Länge des Wortes ist.*

Beweis. (Skizze)

Sei $K = (\Sigma, S, \Gamma, \delta, s_0, \perp, F)$ ein DPDA, der L akzeptiert und sei $w \in L$.

Solange w nicht ganz gelesen wurde, muss nach endlich vielen ϵ -Transitionen das nächste Eingabezeichen gelesen werden.

Der Zeit-Aufwand für jeden Konfigurationsübergang kann durch eine Konstante abgeschätzt werden. $\square$

Satz. *Für $DPDA_{\Sigma}$ gelten folgende Abschlusseigenschaften:*

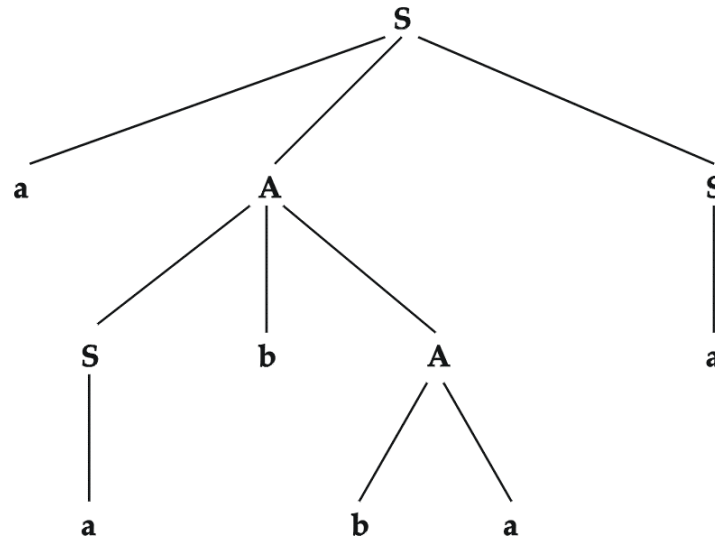
1. $REG_{\Sigma} \subset DPDA_{\Sigma}$
2. Aus $L \in DPDA_{\Sigma}$ folgt $\Sigma^* \setminus L \in DPDA_{\Sigma}$
3. Aus $L_1 \in DPDA_{\Sigma}$ und $L_2 \in REG_{\Sigma}$ folgt $L_1 \cap L_2 \in DPDA_{\Sigma}$
4. $DPDA_{\Sigma}$ ist nicht abgeschlossen gegenüber Durchschnitt, Vereinigung, Konkatenation, Kleene-Stern-Produkt.

Ableitungsbaum und Front

Sei T_G der **Ableitungsbaum** einer Grammatik $G = (\Sigma, N, P, S)$, dann ist die Zeichenkette, die sich durch Konkatination der Blattmarkierungen von links nach rechts ergibt die **Front** von T_G .

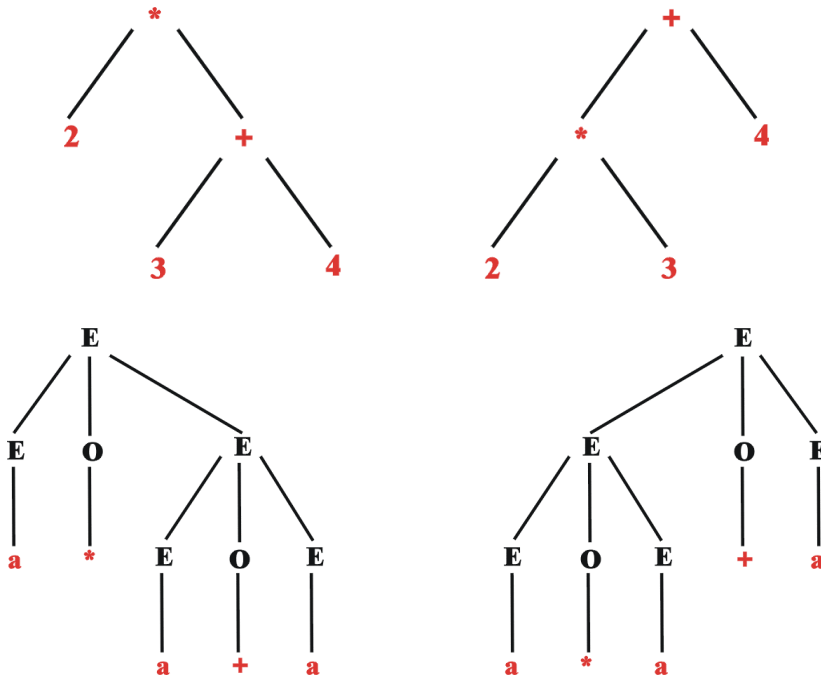
Satz. Sei G eine kontextfreie Grammatik mit Startsymbol S , dann gibt es eine Ableitung $S \Rightarrow^* \omega$ genau dann, wenn es einen Ableitungsbaum über G gibt, dessen Front gleich ω ist.

Bsp.: Ableitungsbaum für $w = aabbaa$ und $P = (S \rightarrow aAS \mid a, A \rightarrow SbA \mid SS \mid ba)$, entsprechend einer Linksableitung oder Rechtsableitung:



Mehrdeutige Grammatiken

Mehrdeutige Grammatiken sind oft **unerwünscht**, weil sie **unterschiedliche Interpretationen** eines Wortes implizieren, etwa $2 * 3 + 4 = 14$ oder $2 * 3 + 4 = 10$:



Noch ein Bsp.: **if** ($b1$) **then** **if** ($b2$) **then** A **else** B

Oft ist die Sprache einer mehrdeutigen Grammatik aber nicht inhärent mehrdeutig.
Es kann also eine **äquivalente nicht mehrdeutige Grammatik** angegeben werden.

Beispiel für Eindeutige Grammatik

$G_{Arith} = (\{a, (,), +, -, *, /\}, \{E, O\}, P, E)$ mit

$P = \{E \rightarrow a \mid EOE \mid (E), O \rightarrow + \mid - \mid * \mid /\}$ erzeugte arithmetische Ausdrücke mehrdeutig.

Für **eindeutige Erzeugung** werden weitere Nichtterminalzeichen benötigt:

- F (Faktor) beschreibt atomare Ausdrücke, geklammert oder eine Zahl
- T (Term) steht für eine Folge von Faktoren, die mit $*$ oder $/$ verknüpft sind
- E (Expression) steht für eine Folge von durch $+$ oder $-$ verbundenen Termen

$G_{Arith}^{eind} = (\{a, (,), +, -, *, /\}, \{E, F, T\}, P, E)$ ist eindeutig mit

$$\begin{aligned} P = \{ & E \rightarrow E + T \mid E - T \mid T, \\ & T \rightarrow T * F \mid T / F \mid F, \\ & F \rightarrow (E) \mid a \} \end{aligned}$$

Bsp.: $E \Rightarrow E + T \Rightarrow T + T \Rightarrow T * F + T \Rightarrow F * F + T \Rightarrow$
 $a * F + T \Rightarrow a * a + T \Rightarrow a * a + F \Rightarrow a * a + a$

Die Grammatik führt direkt zu einem Programm zur Auswertung arithmetischer Ausdrücke.